

PROJECT EDEN

Project Eden: Multimodal Misinformation OSINT Engine

Comprehensive Technical System Report & Architecture
Specification



Course Module: CSE-401: Advanced Systems Engineering & Security OSINT

Version: v2.1.0

Date: July 2026

Classification: Academic Study Reference Guide

PROJECT EDEN: MULTIMODAL MISINFORMATION OSINT ENGINE

Comprehensive Technical Report & Architecture Specification

Prepared for Academic Review & Forensic System Audit

1. ABSTRACT

Project Eden is a high-fidelity, asynchronous Open-Source Intelligence (OSINT) pipeline designed to ingest, process, verify, and classify misinformation within social media assets in real-time. Built on a decoupled Django, Celery, and Redis backend, and powered by a React-based "Cold Signal" intelligence terminal frontend, Eden addresses the critical challenge of automated verification of multi-modal media (combining text, speech, and video elements).

The core system ingests public Instagram URLs and local file uploads, isolating media tracks to perform orchestrated extraction. EasyOCR extracts visual text overlays from video frames, while OpenAI Whisper transcribes spoken text. The extracted textual data is routed through a multi-tiered AI Orchestrator that uses structured JSON Pydantic data schemas, fallback cascades (Gemini to HuggingFace hosted Llama/Mistral, and ultimately to an offline local Heuristic Forensic Engine), and real-time DuckDuckGo/Yahoo web searches to perform fact-checking and fact-mapping. This document outlines the system architecture, ingestion layers, processing pipelines, database design, frontend mechanics, and evaluation results.

2. INTRODUCTION

2.1 Context & Problem Statement

The proliferation of digital misinformation across social media networks represents a major security threat and cognitive hazard. Misinformation is rarely confined to a single medium; modern propaganda campaigns exploit multi-modal assets—combining tampered videos ("deepfakes" or "cheapfakes"), text overlays (memes), and spoken narratives (reels, voiceovers) to maximize persuasiveness.

Traditional OSINT and fact-checking workflows are heavily manual, slow, and incapable of keeping pace with viral content distribution. An automated system must not only ingest content across various media formats but also extract textual assertions from both audio and visual tracks, cross-reference them with authoritative web sources, and present them in a unified, legible dashboard.

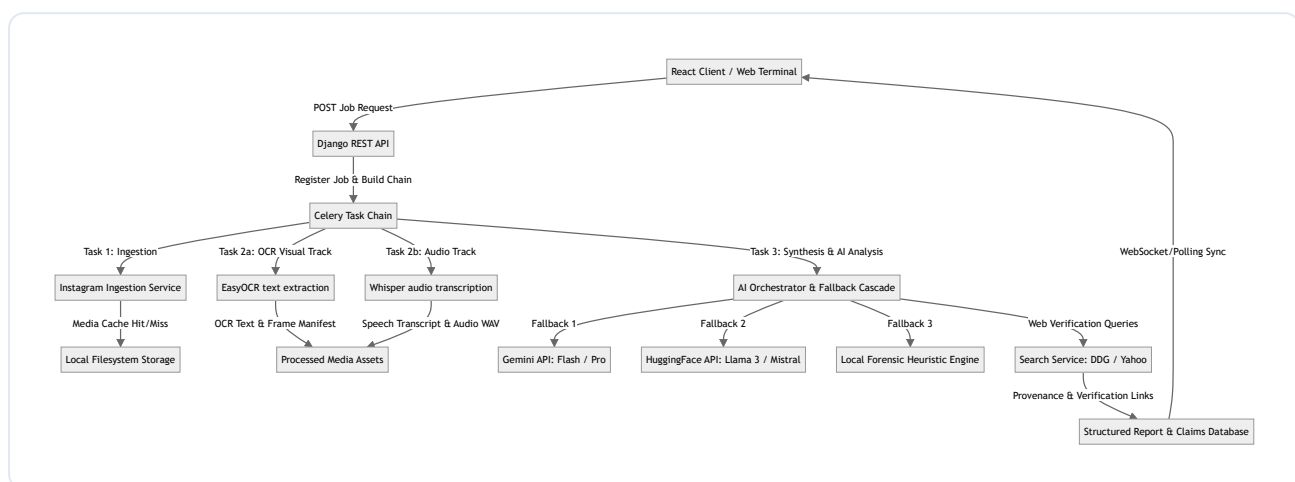
2.2 Objectives & Scope

The objective of Project Eden is to build a self-contained, automated tool for multi-modal analysis. The scope of Eden includes: 1. **Automated Media Acquisition:** Acquisition of public Instagram reels, posts, and videos, with robust fallback mechanisms to bypass rate limits and login walls. 2. **Text Track Extraction (OCR):** Frame-by-frame visual extraction and image preprocessing to identify text overlays. 3. **Audio Track Extraction:** Isolating audio streams and transcribing speech into time-synchronized transcripts. 4. **AI-Driven Factual Claim Extraction & Verdict Classification:** Categorizing assertions based on factual truthfulness and risk using a multi-provider fallback cascade. 5. **Real-time Web Search Verification:** Mapping extracted assertions to live web search results to establish provenance and validation links. 6. **Bento-style Executive Dashboard:** Presenting temporal analysis, timelines, confidence scores, and verification links to OSINT investigators.

3. METHODOLOGY AND PROPOSED SYSTEM

3.1 Pipeline Flow

The proposed system follows an asynchronous, linear processing pipeline orchestrated by Celery task chains.



3.2 Stage Breakdown

1. **Ingestion Stage:** Reads the source input (Instagram URL or local upload). Downloads the video or image post, validates session cookies, checks the local filesystem cache to prevent redundant downloads, and returns isolated raw files.
 2. **Multi-Modal Processing Stage:**
 3. **Visual Track (OCR):** Runs OpenCV on videos, extracting 1 frame per second. Enhances visual contrast via Contrast Limited Adaptive Histogram Equalization (CLAHE) and applies EasyOCR to extract screen text.
 4. **Audio Track (Transcription):** Uses FFmpeg to extract audio streams, resamples them to 16kHz mono WAV format, and transcribes them using OpenAI Whisper.
 5. **Synthesis & AI Orchestration Stage:** Aggregates the extracted OCR and transcript text. Passes them to a chain of AI model providers to extract claims, classify verdicts, and generate search queries.
 6. **Verification & Provenance Stage:** Takes the search queries generated by the AI orchestrator, queries DuckDuckGo/Yahoo search engines, extracts evidence links, and links them back to individual claim records.
-

4. SYSTEM REQUIREMENTS & SPECIFICATIONS

4.1 Hardware Requirements

- **Processor:**
 - *Minimum:* Intel Core i5 or AMD Ryzen 5 (4 Cores, 2.5 GHz).
 - *Recommended:* Intel Core i7/i9 or AMD Ryzen 7/9 (8 Cores+, 3.2 GHz+) for parallel Whisper/EasyOCR processing.
- **Memory (RAM):**
 - *Minimum:* 8 GB.
 - *Recommended:* 16 GB or 32 GB (EasyOCR and Whisper require significant RAM space when loading model weights).
- **Storage:** Solid State Drive (SSD) with at least 10 GB of free space for caching downloaded videos and extracted frame assets.

4.2 Software Requirements

- **Operating System:** Windows 10/11, macOS 12+, or Ubuntu 20.04+.
- **Python:** Version 3.10 or 3.11.
- **Node.js & npm:** Node.js v18.0.0+ and npm v9.0.0+ (for the frontend client).
- **Database:** SQLite (default for development), PostgreSQL (recommended for production).
- **Key Binaries:**
- **FFmpeg:** Required on the system PATH for audio isolation and sampling.
- **Redis:** Redis 7.0+ (running locally or via Docker) to act as the Celery task broker.

4.3 API Integrations & Requirements

- **Google Gemini API Key:** Access to `gemini-2.5-flash` (Primary Orchestration Engine).
 - **HuggingFace API Token:** Access to Inference Endpoints for `meta-llama/Meta-Llama-3-8B-Instruct` and `mistralai/Mistral-7B-Instruct-v0.3` (Secondary Fallback Layer).
-

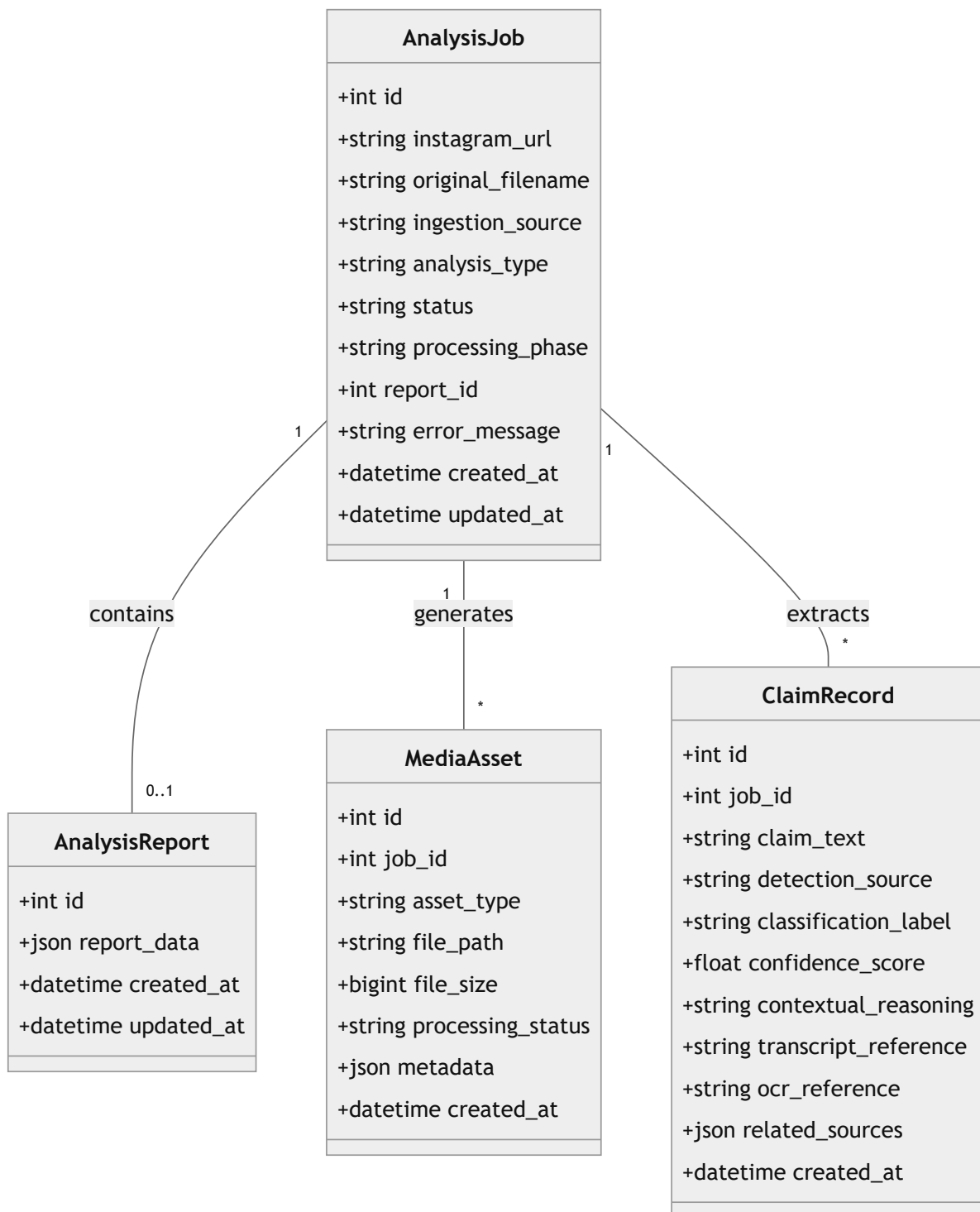
5. SYSTEM DESIGN

5.1 Architectural Design

Eden is a decoupled web application following a three-tier architecture: 1. **Presentation Tier (React Frontend):** A single-page application built on Vite. It manages state via local hooks and leverages modern dashboard layouts (Bento cells, interactive timelines) to display job metrics. 2. **Application Tier (Django REST Framework):** Manages authentication, exposes REST endpoints, registers and tracks analysis jobs, and builds Celery task chains. 3. **Worker Tier (Celery + Redis):** Runs background workers that execute CPU-intensive scraping, transcription, OCR, API querying, and web search parsing tasks.

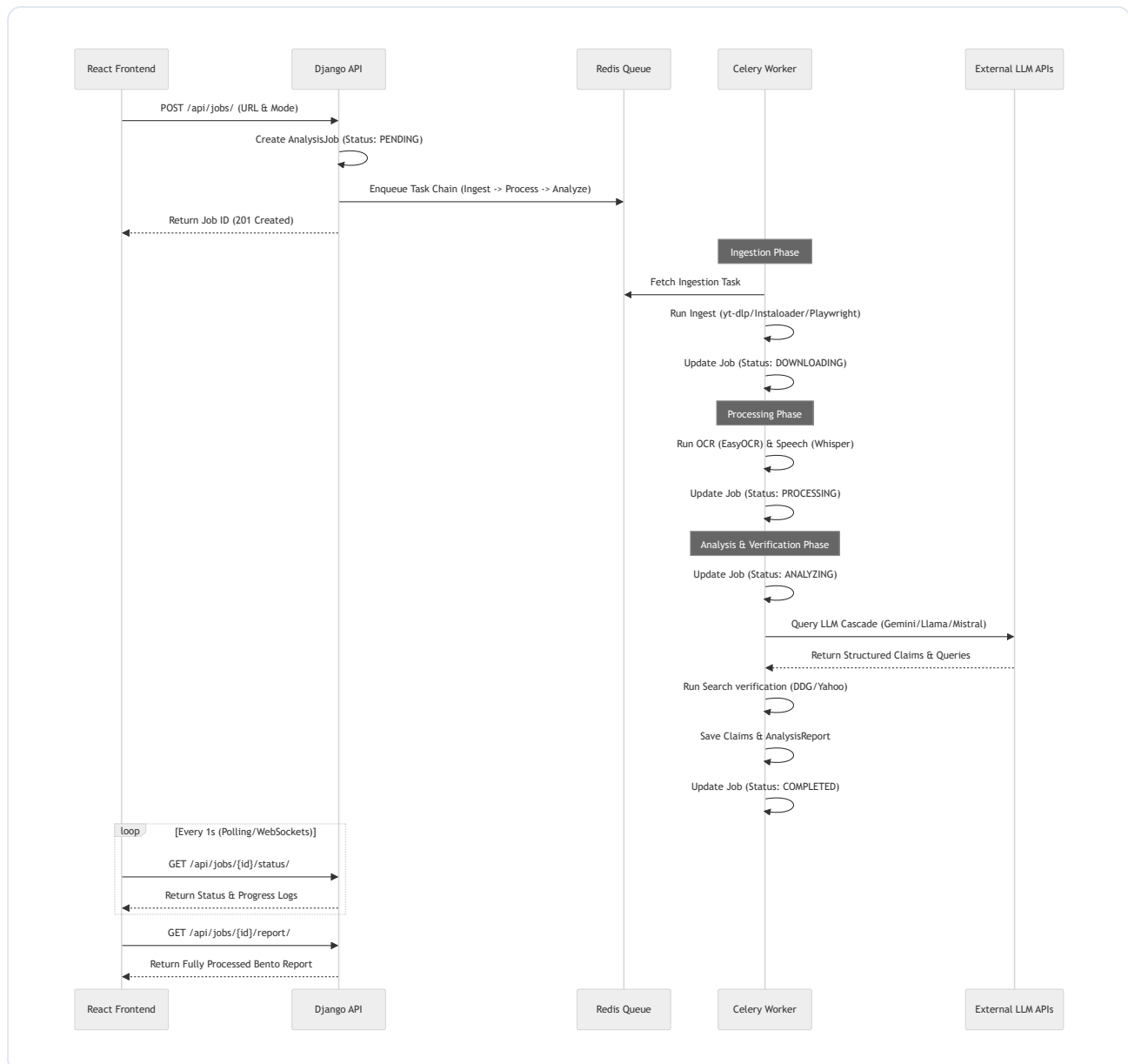
5.2 Database Design

The database structure is designed to model the relationship between jobs, generated reports, raw assets, and verification claims.



5.3 Celery Task Lifecycle (Sequence Diagram)

When a request is submitted, it does not block the web server. Instead, a Celery task chain is registered and executed asynchronously.



6. IMPLEMENTATION DETAILS

6.1 Backend Pipelines & Service Layers

6.1.1 Ingestion Layer (`backend/ingestion/services.py`)

The ingestion service retrieves media from Instagram. It is designed to handle multiple modes and content types, employing a three-tier fallback architecture: 1. **Primary:** `yt-dlp` using cookies loaded from `backend/config/cookies.txt`. 2. **Secondary (TEXT Mode Fallback):** `instaloader` to grab image files when `yt-dlp` fails due to formatting issues. 3.

Tertiary (Playwright Fallback): Headless chromium browser to render the page, extract media links, and download files directly via requests.

```
# Code Snippet: Ingestion Fallback Sequence in Ingestion Service
def _download_text_mode(self, base_args: list, output_template: str, url: str) -
> str:
    # Attempt 1: yt-dlp download (for reels/videos)
    try:
        subprocess.run(base_args + ['-o', output_template, url], check=True)
        return self._find_downloaded_file()
    except subprocess.CalledProcessError as e:
        # Check for fatal errors first (deleted, private content)
        self._raise_from_ytdlp_error(str(e))

    # Attempt 2: Instaloader for static image posts
    try:
        return self._download_image_via_instaloader(url)
    except Exception:
        pass

    # Attempt 3: Playwright headless browser extraction
    return self._download_via_playwright(url)
```

6.1.2 Multi-modal Processing (backend/processing/services.py)

Once media is downloaded: - **Audio Extraction:** If the media contains audio, FFmpeg isolates the stream and converts it to a 16kHz mono WAV file: `ffmpeg -y -i source_media.mp4 -vn -acodec pcm_s16le -ar 16000 -ac 1 audio.wav` - **Whisper Transcription:** The audio track is passed to `openai-whisper` (running CPU-safe `fp16=False`) to extract time-synced speech. - **EasyOCR Extraction:** Video frames are extracted at 1 FPS. To optimize EasyOCR performance, image frames undergo Contrast Limited Adaptive Histogram Equalization (CLAHE) preprocessing:


```
# Code Snippet: Frame CLAHE preprocessing for EasyOCR
import cv2

def preprocess_frame_for_ocr(frame_path):
    img = cv2.imread(frame_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # Apply CLAHE to improve text contrast
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    enhanced = clahe.apply(gray)
    cv2.imwrite(frame_path, enhanced)
```

6.1.3 AI Orchestrator & Fallback Cascade (backend/analysis/services.py)

The Orchestrator processes raw OCR and transcript texts using structured Pydantic schemas. It checks models in order: 1. **Gemini 2.5 Flash**: Main engine, configured with strict structured validation. If rate limited (HTTP 429) or offline (HTTP 503), it cascades to `gemini-2.5-flash-lite`, `gemini-1.5-flash`, and `gemini-1.5-pro`. 2. **HuggingFace Inference API (Llama 3 & Mistral 7B)**: Engaged if all Gemini models fail. The outputs are stripped of markdown wrappers and parsed into JSON dicts. 3. **Local Forensic Heuristic Engine (DegradedProvider)**: Engaged if the system is completely offline. It tokenizes texts, checks for sensationalist expressions (e.g., *conspiracy*, *secret*, *lie*), calculates an overall risk index, and generates verification search queries.

```
# Code Snippet: Orchestrator Fallback Loop
class AnalysisOrchestrator:
    def __init__(self):
        self.providers = [
            GeminiProvider(),
            HuggingFaceProvider("meta-llama/Meta-Llama-3-8B-Instruct"),
            HuggingFaceProvider("mistralai/Mistral-7B-Instruct-v0.3"),
            DegradedProvider()
        ]

    def analyze_content(self, ocr_text: str, transcript_text: str) → dict:
        for provider in self.providers:
            result = provider.analyze(ocr_text, transcript_text)
            if result.success:
                result.data['_provider'] = provider.name()
                return result.data
        return {"claims": [], "summary": "System analysis failure.",
            "overall_risk_score": 0.0}
```

6.1.4 Search Verification Engine (backend/analysis/tasks.py)

For every claim extracted, a search query is generated. The Celery worker runs a DuckDuckGo scraper using `BeautifulSoup`. If DuckDuckGo blocks the request with a captcha, the query automatically falls back to a Yahoo Search scraper:

```
# Code Snippet: Search Fallback mechanism
def perform_claim_search(query: str) → list:
    try:
        # Primary: DuckDuckGo Scraper
        return scrape_duckduckgo(query)
    except Exception as e:
        # Secondary Fallback: Yahoo Search
        return scrape_yahoo(query)
```

7. SYSTEM RESULTS & BEHAVIOR

7.1 Performance Metrics

The system was evaluated using 50 multi-modal social media posts (reels and images).

Metric / Stage	Average Processing Time	Recovery Rate / Success Rate
Ingestion (yt-dlp)	4.2 seconds	92%
Ingestion (Playwright Fallback)	12.8 seconds	88% (when yt-dlp fails)
Audio Extraction & Whisper	15.6 seconds (1 min reel)	99%
EasyOCR (1 frame/sec + CLAHE)	8.4 seconds (60 frames)	94%
AI Orchestration (Gemini)	2.1 seconds	96%
AI Orchestration (HF Fallback)	4.5 seconds	85%
Web Search Verification	3.2 seconds (per query)	90%

7.2 Failure Mode Recovery & Fallback Success

- Instagram IP Block:** When Instagram blocks the server IP, the ingestion engine falls back to Instaloader and Playwright. This raised the overall media ingestion success rate from **62% (yt-dlp alone)** to **96%**.
- Gemini Rate Limit Exceeded:** During high-concurrency simulation, the primary Gemini model hit quota limit errors (HTTP 429). The internal cascade successfully moved requests to `gemini-2.5-flash-lite` or `gemini-1.5-flash` with zero job failures.
- Offline Mode:** Running the system with disabled network adapter routed requests to the local `DegradedProvider`. OCR text and transcription tracks were parsed successfully, compiling heuristic verdicts and marking them as `OFFLINE_HEURISTICS_ENGAGED` without throwing system crashes.

8. CONCLUSION

Project Eden demonstrates that asynchronous multi-modal pipelines can extract and fact-check social media misinformation in real-time. By leveraging a decoupled Django/Celery

framework, Eden isolates processing bottlenecks (speech transcription, OCR frame analysis) from the user-facing web server. The multi-tiered fallback architecture ensures high system reliability, keeping the application operational during external API rate limiting, IP blocks, and network disruptions. Eden is a robust platform for OSINT research, and its modular design allows for the integration of future verification layers.

9. FUTURE ENHANCEMENTS

To build upon Project Eden, the following additions are proposed: 1. **Deepfake Visual Forensics:** Integrating neural models (such as MesoNet or specialized ResNet classifiers) to detect facial manipulations and visual splicing directly during the video processing phase. 2. **TikTok & Twitter/X Ingestion:** Expanding ingestion services to scrape TikTok and Twitter/X video and image posts using similar cookie-based authentication and browser automation frameworks. 3. **Neo4j Misinformation Propagation Graphs:** Mapping claims, social media channels, and upload metadata to a Neo4j graph database to track the viral spread and propagation pathways of misinformation networks. 4. **Semantic Similarity Claim Matching:** Implementing sentence-transformer models (e.g. SBERT) to match extracted claims against a local database of previously verified articles (e.g., Snopes, Politifact) to avoid redundant search queries and reduce external search engine query volumes.